

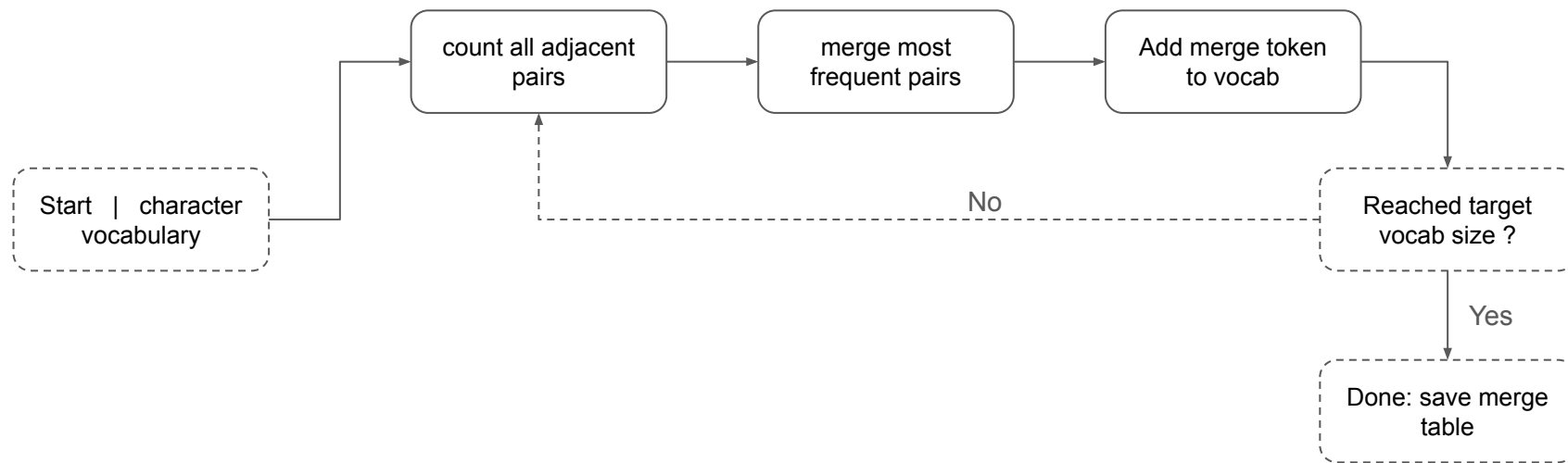
Tokenizers

AI Engineering From Scratch #10

Byte Pair Encoding

- A greedy compression algorithm.
- Start with individual characters:
 - Count every adjacent pair in the training corpus.
 - Merge the most frequent pair into a new token.
 - Repeat until you reach your target vocabulary size.
- Used in GPT-2, GPT-3, GPT-4, ...

Byte Pair Encoding



WordPiece

- Different merge strategy:
 - BPE merge criterion: $\text{count}(A, B)$
 - WordPiece criterion: $\text{count}(A, B) / (\text{count}(A) * \text{count}(B))$
- Instead of raw frequency, it asks:
 - “Is this pair occurring together more often than we’d expect from the individual frequencies?”
 - →it means: if u appears almost everywhere, then (u, g) may get lower score because u is not very informative.
- Uses “##” to present the continuation subwords.
- Used by BERT.

Sentencepiece

- Treats input as a raw stream of Unicode characters, including whitespace.
- Supports two algorithms:
 - BPE mode: same merge logic as standard BPE, applied to raw character sequences.
 - Unigram mode: starts with a large vocabulary and iteratively removes tokens that least affect to the overall likelihood. **The reverse of BPE - prune instead of merge.**
- Sentencepiece BPE used by llama-2, llama 3,...
- Sentencepiece Unigram used by T5.

Vocab size tradeoffs

Large vocab:

- Fewer tokens per text.
- Shorter sequences.
- Larger embedding matrix
- Faster inference.

Vocab size tradeoffs

Small vocab:

- More tokens per text.
- Longer sequences.
- Smaller embedding matrix.
- Better rare-word handling.